A Comparative Study of SAT Encodings and Exact Solvers for the N-Queens Problem

Nguyen Nhat Minh[a,*]

[a]Faculty of Information Technology, VNU University of Engineering and Technology, Vietnam National University, Hanoi, Vietnam

ARTICLE INFO

Keywords:
N-Queens
Boolean satisfiability
cardinality encoding
constraint programming
mixed integer programming
exact algorithms

ABSTRACT

The N-Queens problem asks for a placement of N queens on an $N \times N$ chessboard such that no two queens share a row, column, or diagonal. This study implements a SAT-based solver and compares pairwise, sequential-counter, and bitwise at-most-one encodings, then evaluates a provisional SAT baseline against CP-SAT and mixed-integer programming solvers. Repeated local runs show that Pairwise has the lowest median SAT runtime at three of the four repeatedly measured sizes from $N = 8$ to $N = 64$; Sequential Counter is fastest at $N = 16$. A matched three-run cross-solver batch covers ten board sizes up to $N = 128$. All five methods solve every tested case, and all 150 results pass an independent validator. In this batch, SAT has the lowest median at $N = 8, 16$, Gurobi at $N = 24, 31, 45$ and narrowly at $N = 64$, CPLEX MIP at $N = 32$, and CPLEX CP at $N = 44, 100, 128$. At $N = 64$, the Gurobi and CPLEX CP medians are 0.0359984 s and 0.0361838 s, respectively. The results are a repeated local comparison, with conclusions limited by three runs per configuration.

1. Introduction

The N-Queens problem is a classical constraint satisfaction problem in which N queens must be placed on an $N \times N$ board so that no two queens attack one another. Although the problem has a simple statement, it is a useful benchmark for comparing exact solving paradigms because the number of possible placements grows rapidly with N and the constraints can be represented using several modelling formalisms.

This study focuses on a SAT formulation and compares it with exact methods based on constraint programming and mixed integer programming. The SAT part is implemented with PySAT and the Glucose3 solver [4]. Three at-most-one (AMO) encodings are evaluated while the underlying SAT solver and N-Queens constraints remain fixed. This design isolates the effect of the cardinality encoding [6].

2. Related Work

SAT solvers operate on Boolean formulas in conjunctive normal form (CNF), so cardinality constraints such as "at most one" require a CNF encoding. Pairwise clauses provide a direct encoding, while sequential counters introduce auxiliary variables to represent partial counts. Sinz presents a systematic treatment of cardinality encodings, including sequential-counter constructions and their size trade-offs [6]. This study uses the pairwise, sequential-counter, and bitwise encodings exposed by PySAT's cardinality interface; its focus is their measured behavior in one N-Queens formulation, rather than a new encoding.

Constraint programming expresses the same structure through global constraints. Here, the row-to-column variables use all-different constraints on columns and on the two diagonal expressions. CP-SAT provides an integer and Boolean constraint-solving interface, while CPLEX CP Optimizer provides a CP implementation [2, 5]. The MIP models instead use binary board-position variables and linear constraints, solved with CPLEX and Gurobi [1, 2]. PySAT supplies the common interface to the Glucose3 SAT oracle used in the encoding comparison [4]. These systems represent different modelling and solving approaches; the present benchmark compares concrete implementations and does not claim a general ranking of the paradigms.

The contributions of this work are:

- a reusable SAT encoding architecture for N-Queens;

*Corresponding author.
23021631@vnu.edu.vn (N.N. Minh)
ORCID(s):

- an implementation of pairwise, sequential-counter, and bitwise AMO encodings;
- implementations of CP-SAT, CPLEX CP, CPLEX MIP, and Gurobi MIP models;
- a common validator and benchmark result schema;
- a repeated comparison of model size, execution time, solver status, and runtime performance.

3. Problem Definition and Mathematical Models

Let $x_{r,c}$ denote whether a queen is placed at row r and column c, where $0 \le r, c < N$. The N-Queens constraints require exactly one queen in each row and column and at most one queen on each diagonal.

For SAT and MIP, $x_{r,c}$ is a Boolean or binary decision variable. The row constraints are:

$$\sum_{c=0}^{N-1} x_{r,c} = 1 \qquad \forall r. \tag{1}$$

The column constraints are:

$$\sum_{r=0}^{N-1} x_{r,c} = 1 \qquad \forall c. \tag{2}$$

For each main diagonal and anti-diagonal, the corresponding variables satisfy an at-most-one constraint. Main diagonals are grouped by $r - c$ and anti-diagonals by $r + c$.

More explicitly, for every feasible diagonal index d and s, the linear formulation imposes:

$$\sum_{\substack{0 \le r,c < N \\ r-c=d}} x_{r,c} \le 1 \qquad \forall d \in \{-(N-1), \dots, N-1\}, \tag{3}$$

and

$$\sum_{\substack{0 \le r,c < N \\ r+c=s}} x_{r,c} \le 1 \qquad \forall s \in \{0, \dots, 2N-2\}. \tag{4}$$

Diagonal groups containing a single board position need no explicit constraint. Together, the row, column, and diagonal constraints give the binary SAT/MIP model; the row-indexed CP model expresses the same restrictions through three global all-different constraints.

The CP and CP-SAT models use N integer variables q_r, where q_r is the column assigned to row r. They enforce:

$$\text{AllDifferent}(q_r), \quad \text{AllDifferent}(q_r + r), \quad \text{AllDifferent}(q_r - r). \tag{5}$$

4. SAT-Based Approach

4.1. CNF Encoding

Each board position is mapped to a SAT variable using:

$$v(r, c) = rN + c + 1. \tag{6}$$

Exactly-one constraints are decomposed into an at-least-one clause and an AMO constraint. The implementation uses a shared PySAT `IDPool` whose auxiliary variables start after the N^2 original board variables. This prevents auxiliary-variable collisions across rows, columns, and diagonals.

4.2. AMO Encodings

For an AMO constraint over $x_1, \ldots, x_m$, the pairwise encoding adds:

$$\bigwedge_{1 \le i < j \le m} (\neg x_i \vee \neg x_j). \tag{7}$$

For example, for three candidate squares a, b, c in one row, it adds $(\neg a \vee \neg b) \wedge (\neg a \vee \neg c) \wedge (\neg b \vee \neg c)$. Together with the at-least-one clause $(a \vee b \vee c)$, this says exactly one candidate is occupied. Pairwise introduces no auxiliary variables and uses $\binom{m}{2}$ AMO clauses.

The sequential-counter encoding introduces prefix indicators s_i, where s_i records that at least one of $x_1, \ldots, x_i$ is true. For an AMO constraint, the clauses propagate each true input through the prefix and forbid a later input when the previous prefix is already true. One standard form is $(\neg x_1 \vee s_1)$; for $2 \le i < m$, $(\neg x_i \vee s_i)$, $(\neg s_{i-1} \vee s_i)$, and $(\neg x_i \vee \neg s_{i-1})$; and the final clause $(\neg x_m \vee \neg s_{m-1})$. Thus the encoding uses auxiliary state to replace the quadratic pairwise exclusion pattern with a linear-size construction for each AMO group.

The bitwise encoding assigns each input a distinct binary code using $k = \lceil \log_2 m \rceil$ selector bits. For each input x_i and bit position, a clause makes x_i imply the corresponding bit value of its code. If two inputs were true, the selector bits would have to equal two distinct codes, which is impossible. This uses selector variables and roughly mk implication clauses for a group, rather than one clause for each pair. In the implementation, each AMO group receives its own auxiliary variables from a shared ID pool. The encodings are generated by PySAT's cardinality API [3].

Table 2 counts the N^2 board variables plus all auxiliary variables, and all clauses in the complete formula. At $N = 64$, for example, Pairwise has 4,096 variables and 428,864 clauses; Sequential Counter has 20,094 variables and 47,752 clauses; Bitwise has 6,132 variables and 95,572 clauses. These totals combine row and column exactly-one constraints with diagonal AMO constraints, so they are larger than the counts for a single AMO group. Across the reported sizes, Pairwise minimizes variables, Sequential Counter minimizes clauses, and Bitwise falls between them. These structural counts describe formula size, not solver effort by themselves.

4.3. Solution Decoding and Validation

If the SAT instance is satisfiable, the model is decoded into the representation $q_r = c$. The decoded placement is then checked independently. The validator verifies the board size, column bounds, column uniqueness, main-diagonal uniqueness, and anti-diagonal uniqueness. This validation is separate from the solver model implementation.

The SAT solver tests also check the expected small-instance boundary for each of the three encodings, as summarized in Table 1. These unit tests complement the larger satisfiable benchmark instances.

Table 1
Expected SAT status for small N-Queens instances, checked for each SAT encoding.

N	Expected status
1	SAT (validated)
2	UNSAT
3	UNSAT
4	SAT (validated)

5. Compared Exact Solvers

The study compares the SAT solver with four exact alternatives. OR-Tools CP-SAT [5] uses the integer-variable formulation and records conflicts and branches where available. CPLEX CP Optimizer [2] uses the same high-level all-different formulation. CPLEX MIP and Gurobi MIP [1] use binary board-position variables and linear row, column, and diagonal constraints.

At a high level, the SAT backend searches for a satisfying assignment to the CNF formula using conflict-driven Boolean search [4]. The CP models propagate domain restrictions from the all-different constraints and branch on integer variables [2, 5]. The MIP solvers apply linear-programming relaxations and branch-and-bound, with solver-managed presolve and other default techniques [1, 2]. These descriptions identify the solving paradigms; internal heuristics and detailed search settings are controlled by the respective solver libraries and are not modified by this study.

5.1. Implementation and Solution Validation

The implementation is organized by solving paradigm. The SAT encoder maps each board square to one of the first N^2 Boolean variables and uses a shared auxiliary-variable pool for non-pairwise cardinality encodings. It adds exactly-one constraints for rows and columns, followed by at-most-one constraints for each diagonal group. The SAT solver passes the resulting CNF to Glucose3, reads the returned model, and decodes the selected square in each row into a column vector q.

The CP-SAT and CPLEX CP implementations create one bounded integer variable $q_r \in \{0, \dots, N-1\}$ per row and post the three all-different constraints. The CPLEX MIP and Gurobi MIP implementations create binary variables $x_{r,c}$, add the row and column equalities and diagonal inequalities above, and extract the selected column from each row after optimization. Each implementation maps the solver's native status to a common SAT, UNSAT, or UNKNOWN status where applicable. A shared validator then independently checks that the decoded solution has one queen per row, uses each column once, and has no repeated main- or anti-diagonal index. This check is performed outside the solver model and is reported separately from the solver status.

The benchmark runner normalizes all solver outputs into a common schema containing status, validity, model-size fields where available, build or encoding time, solve time, total time, raw status, and error information.

6. Experimental Setup

6.1. Hardware and Software

The benchmarks were run on a Dell Inspiron 15 3511 with an 11th-generation Intel Core i5-1135G7 processor (4 cores, 8 logical processors) and 8 GB RAM, using Windows 11 Pro (build 22631) and Python 3.11.8. The installed software versions were PySAT 1.9.dev16, OR-Tools 9.15.6755, Gurobi 13.0.3, DOcplex 2.32.264, and CPLEX 22.2.0. CPLEX MIP and CP Optimizer passed separate 1,100-variable full-version license checks. Gurobi passed a 2,100-variable academic-license check. The full benchmark set reported here was rerun in the current environment. The repository's last recorded full test-suite run passed 43 tests; pytest was not rerun as part of this benchmark refresh.

6.2. Benchmark Protocol

The repeated SAT encoding experiment covers $N = 8, 16, 32, 64$, with five runs for $N = 8, 16, 32$ and three runs for $N = 64$. Historical exploratory CSVs contain one run per encoding at $N = 96$ and $N = 128$. No row in those archived files is marked TIMEOUT. At $N = 128$, Pairwise took 8.84 s, Bitwise 0.97 s, and Sequential Counter 3,045.89 s; this one Sequential Counter run exceeded the 600-second cutoff used in the later benchmark protocol but is recorded as SAT. These single exploratory observations are excluded because they are not repeated measurements under the final protocol, not because all encodings timed out. They also show why Pairwise solving $N = 128$ in the matched solver table is consistent: that table uses Pairwise and is a separate experiment.

The matched cross-solver batch uses Pairwise SAT at $N = 8, 16, 24, 31, 32, 44, 45, 64, 100, 128$, with three runs per method and size. The $N = 31/32$ and $N = 44/45$ pairs retain the earlier CPLEX and Gurobi license-boundary cases; all solvers now solve these sizes with the active licenses. The complete five-method batch was rerun in one session after Gurobi academic-license activation. $N = 8, 16, 24$ provide small and medium reference cases; $N = 64, 100, 128$ extend the comparison to larger instances. A 600-second per-configuration timeout is retained. Timeouts and license rejections are stored as explicit statuses, not treated as runtimes. The tables report medians of three total times; the raw CSV preserves all 150 outcomes.

For each successful run, total time is the sum of model-construction time and solver-call time: encoding plus solve time for SAT, and build plus solve time for CP-SAT, CPLEX CP, CPLEX MIP, and Gurobi MIP. The timer excludes solution decoding and independent validation. The 600-second limit is enforced by the benchmark runner at the process level, while the reported successful-run totals use the timers inside each solver implementation. Each configuration is run three times in the matched comparison, and the reported value is the median of those three totals. Solver-specific default parameters are used; thread counts and random seeds are not explicitly standardized, so the measurements reflect the installed versions' defaults.

Pairwise was selected as the provisional SAT baseline for the cross-solver comparison. In the latest repeated encoding batch, it had the lowest median total time at $N = 8, 32, 64$, while Sequential Counter was fastest at $N = 16$ (0.0093 s versus 0.0169 s for Pairwise). At $N = 8$, Pairwise was fastest at 0.0013 s, with Sequential Counter at 0.0015 s and Bitwise at 0.0020 s. Pairwise is a practical choice for the larger repeatedly measured cases in this dataset, but it does not dominate every size. This choice is limited to the measured PySAT/Glucose3 implementation and does not

establish that Pairwise is best at every size or with every SAT solver. Sequential Counter and Bitwise remain in the encoding comparison; exploratory $N = 96, 128$ observations are excluded because there is only one archived run per encoding and no matched repeated encoding batch.

6.3. Reproducibility

Source code, benchmark outputs, summary tables, and figures are maintained in the project repository:

`https://github.com/nNm205/n-queens-solving`

7. Experimental Results

7.1. SAT Encoding Comparison

Table 2 reports the deterministic size of each complete SAT formula. Variable counts include the N^2 board variables and any encoding auxiliaries; clause counts include the full N-Queens formula. The at-least-one clauses are common to all three encodings, so the size differences arise from their AMO constraints. Pairwise uses the fewest variables, while Sequential Counter produces the fewest clauses at every reported size. Bitwise lies between them on both measures at all four sizes.

Table 2
SAT formula size by encoding. Counts are deterministic for each N.

Encoding	N	Variables	Clauses
Pairwise	8	64	744
Sequential Counter	8	270	600
Bitwise	8	170	724
Pairwise	16	256	6,352
Sequential Counter	16	1,182	2,728
Bitwise	16	568	3,924
Pairwise	32	1,024	52,640
Sequential Counter	32	4,926	11,592
Bitwise	32	1,846	19,796
Pairwise	64	4,096	428,864
Sequential Counter	64	20,094	47,752
Bitwise	64	6,132	95,572

Table 3 reports median total times from the latest repeated local batch. Pairwise is fastest at $N = 8, 32, 64$, while Sequential Counter leads at $N = 16$ (0.0093 s versus 0.0169 s). Bitwise is slower at $N = 16, 32, 64$. The formula-size trade-off is clear: Pairwise uses fewer variables but more clauses, whereas Sequential Counter reduces clauses with additional auxiliary variables. Neither count alone explains runtime. These medians describe one machine and should not be read as universal rankings.

Table 3
Median total time in seconds for the repeated SAT encoding batch.

N	Pairwise	Sequential counter	Bitwise	Repeats
8	0.0013	0.0015	0.0020	5
16	0.0169	0.0093	0.1088	5
32	0.0891	0.5117	16.7465	5
64	0.6686	12.3846	22.6755	3

The exploratory $N = 96, 128$ data are not included in this repeated comparison: the archived data contain only one run per encoding, and the $N = 128$ Sequential Counter run took 3,045.89 s. Pairwise is used provisionally for the cross-solver comparison because it is fastest at three of the four repeated sizes and has substantially lower medians than Sequential Counter at $N = 32$ and $N = 64$. Sequential Counter is faster at $N = 16$, so the measured data do not support a claim that one encoding wins at every size. The clause and variable counts are separate model-size measures; neither alone predicts runtime because propagation strength, clause structure, preprocessing, and the Glucose3 search all contribute.

7.2. Cross-Solver Comparison

Table 4 reports medians from the matched three-run Pairwise comparison. It includes all five methods: SAT, CP-SAT, CPLEX CP, CPLEX MIP, and Gurobi MIP. All 150 solutions passed the independent validator.

Table 4
Median total time (seconds) from three Pairwise cross-solver runs per configuration. Values are rounded to three decimals except the two $N = 64$ medians, shown to six decimals.

N	SAT	CP-SAT	CPLEX CP	CPLEX MIP	Gurobi MIP
8	0.003	0.034	0.059	0.039	0.010
16	0.037	0.074	0.057	0.046	0.046
24	0.172	0.119	0.062	0.059	0.016
31	0.197	0.209	0.090	0.088	0.084
32	0.238	0.215	0.089	0.073	0.079
44	0.528	0.427	0.064	0.122	0.150
45	0.402	0.217	0.047	0.054	0.016
64	0.727	0.389	0.036184	0.085	0.035998
100	2.765	0.979	0.044	0.171	0.059
128	6.183	1.935	0.049	0.312	0.146

The latest matched results show that all five methods solve all ten tested sizes. SAT has the lowest median at $N = 8, 16$; Gurobi at $N = 24, 31, 45, 64$; CPLEX MIP at $N = 32$; and CPLEX CP at $N = 44, 100, 128$. At $N = 64$, Gurobi's median is 0.0359984 s and CPLEX CP's is 0.0361838 s, so Gurobi is narrowly faster in the raw median (by 0.0001854 s, about 0.5

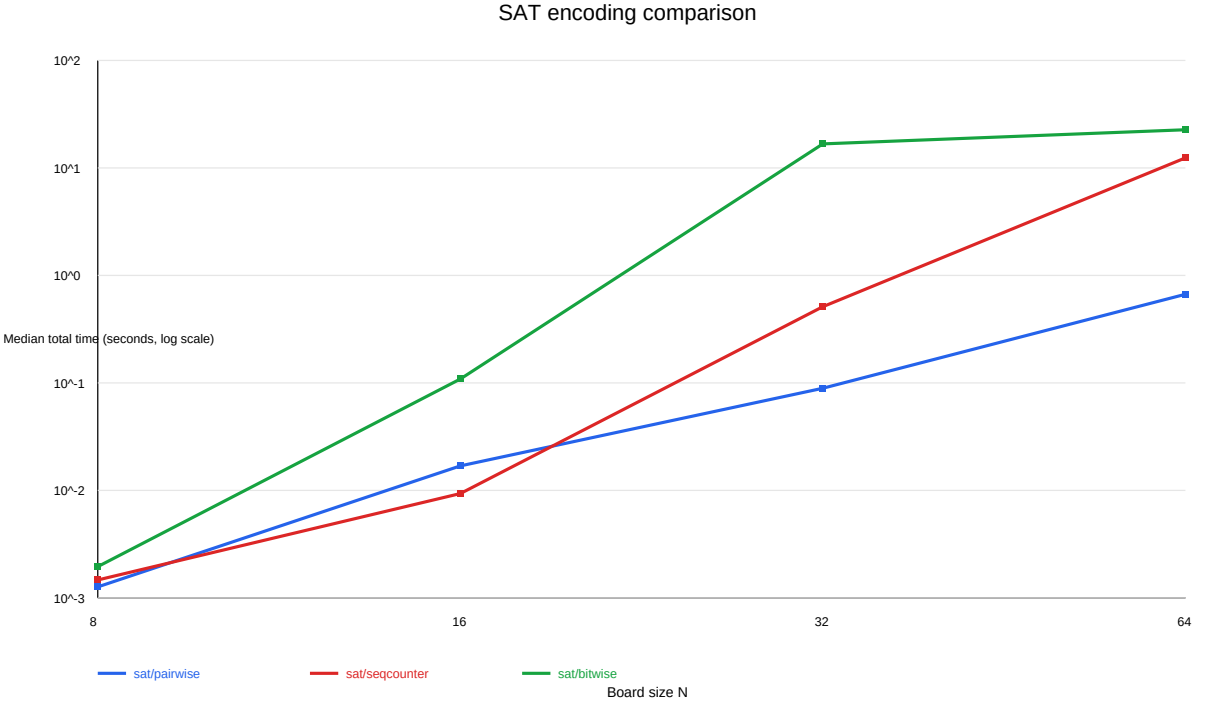


Figure 1: SAT encoding runtime comparison from the repeated local batch.

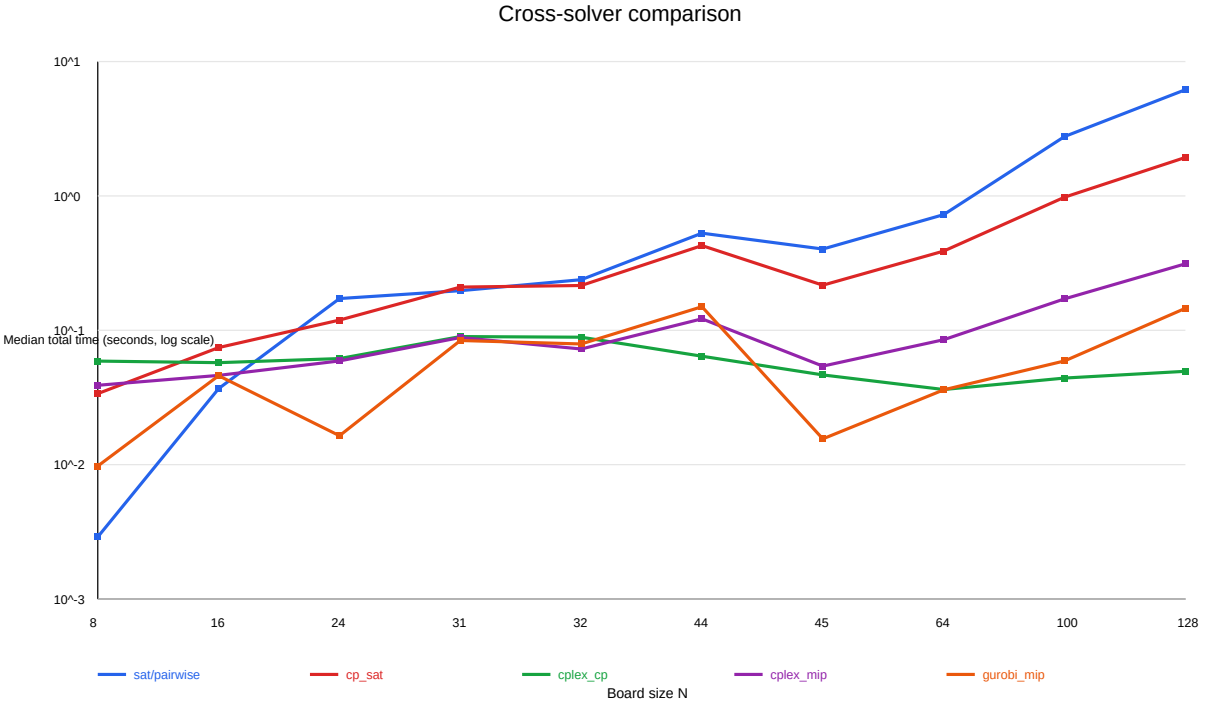


Figure 2: Cross-solver runtime comparison on a logarithmic scale.

8. Limitations and Discussion

The current experiments are limited by the small sample size. CPLEX MIP and CP Optimizer passed 1,100-variable license checks and solved every tested board size through $N = 128$. After academic license activation, Gurobi passed a 2,100-variable license check and solved the previously limited $N = 45$ and larger configurations.

The three-run cross-solver data provide a comparison on one laptop, but it remains a small local sample. The experiment does not control for warm-up, thermal throttling, background load, or variation across machines. All five methods were rerun in the same batch at every tested size from $N = 8$ through $N = 128$.

The SAT encoding experiment holds Glucose3 and the N-Queens formulation fixed while varying only the AMO encoding, so it studies the interaction between encoding structure and this SAT backend. It does not isolate encoding behavior from solver behavior in general. Sequential Counter's smaller clause count does not guarantee a faster solve: its extra variables and propagation structure can alter Glucose3's search. The measured total time includes encoding, and for tiny instances some totals are only milliseconds; fixed startup, Python, and process overhead can therefore form a large share of the measured time.

8.1. Future Work

Further work should increase repetitions and report runtime distributions, control thread count and machine load, compare additional SAT backends, separate model-construction from solve time, and run larger repeated encoding comparisons under one timeout protocol. Repeating the measurements across machines would also help establish how much the observed rankings depend on the local environment.

9. Conclusion

This paper presented a unified implementation for solving N-Queens with SAT, constraint programming, and mixed integer programming. The SAT implementation supports three AMO encodings and includes independent solution validation. The benchmark infrastructure records common performance and status fields and continues after solver-specific errors.

The repeated encoding batch supports Pairwise as a provisional SAT baseline for this Glucose3 implementation: it has the lowest median at three of the four repeatedly measured sizes, while Sequential Counter is faster at $N = 16$. In the matched cross-solver batch, all five methods solve all ten tested sizes through $N = 128$. SAT records the lowest medians at $N = 8, 16$, Gurobi at $N = 24, 31, 45$, CPLEX MIP at $N = 32$, CPLEX CP at $N = 44, 100, 128$, and Gurobi narrowly at $N = 64$ (0.0359984 s versus 0.0361838 s for CPLEX CP). The $N = 128$ median is 6.183 s for Pairwise SAT and 0.049 s for CPLEX CP, a roughly 126-fold observed difference. A compact CP formulation with global all-different constraints is a plausible contributor, but this experiment cannot establish the cause or a universal advantage. These three-run, single-machine results support a local comparison rather than a general solver ranking. Future comparisons should use more repetitions, controlled execution conditions, multiple SAT backends, and larger repeated encoding batches.

Declaration of Competing Interest

The author declares that there is no known competing financial interest or personal relationship that could have appeared to influence the work reported in this paper.

Data Availability

Source code, benchmark result files, summary tables, and figures are available at: `https://github.com/nNm205/n-queens-solving`

Declaration of Generative AI and AI-Assisted Technologies

During the preparation of this work, the author used ChatGPT by OpenAI to assist with language revision, code organization, and analysis drafting. The author reviewed and edited the generated material and takes full responsibility for the final content.

References

[1] Gurobi Optimization, LLC, 2026. Gurobi Optimizer Reference Manual, version 13.0. https://docs.gurobi.com.

[2] IBM Corporation, 2026. IBM ILOG CPLEX Optimization Studio documentation, version 22.2. https://www.ibm.com/docs/en/icos.

[3] Ignatiev, A., Marques-Silva, J., Morgado, A., 2026. PySAT cardinality encodings api documentation. https://pysathq.github.io/docs/html/api/card.html. Version 1.9.dev16.

[4] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A Python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437.

[5] Perron, L., Didier, F., 2026. CP-SAT v9.15. https://developers.google.com/optimization/cp/cp_solver.

[6] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831. doi:10.1007/11564751_73.